Hospital Chatbot — Documentation

Table of contents

1. Project Overview
2. Project Requirements
3. User Roles
4. Technology Stack
5. System Architecture
6. System Workflow
7. Directory Structure
8. Startup and API Entry Points
9. WhatsApp Message Journey
10. Conversation Flows
11. Booking
12. Reschedule and Cancel
13. Working Hours, Slots, and Holidays
14. Reminders
15. Voice
16. Safety and Handover
17. Data Structure & Database Design
18. Admin Dashboard
19. API Documentation
20. Deployment
21. User Manual
22. Testing
23. Conclusion
24. Document History
25. Appendix
    - A — Tools & Technologies
    - B — API Endpoints
    - C — Environment Variables

1. Project Overview

Hospital Chatbot is a WhatsApp Cloud API backend for clinic appointment operations. Patients can:

- Book, reschedule, and cancel appointments
- Search by specialty or doctor name
- Ask about doctors/specialties
- Send voice notes
- Escalate to reception (handover)

Arabic and English are supported. Clinic staff use a Streamlit admin dashboard against the same PostgreSQL database.

Production entry point: `uvicorn app.main:app`

Interface	Capabilities
Patient (WhatsApp)	Text, buttons, lists, doctor cards, optional Flow, voice
Admin (Streamlit)	Clinic/doctor/hours setup, appointments, conversations, reminders, KPIs, usage

2. Project Requirements

This section describes **what the system must deliver** from a product and user perspective (before technical construction details).

Requirement	Description
Book an appointment	Patient selects specialty or doctor, then date and time, enters patient details, and confirms; system creates the appointment and returns a booking reference.
Reschedule an appointment	Patient selects an upcoming appointment, chooses a new date/time, and confirms; old slot is released and reminders are refreshed.
Cancel an appointment	Patient selects an upcoming appointment and confirms cancellation; slot is released and pending reminders are cancelled.
Search by specialty	Patient browses departments/specialties and then doctors in that specialty.
Search by doctor name	Patient browses a paginated list of active doctors by name and selects one.
Ask about doctors and specialties	Inquiry path to browse doctors/specialties; may continue into booking.
Send voice messages	Patient may send WhatsApp voice notes; system transcribes and routes into the same guided flows when possible.
Arabic and English	Conversation language follows the patient's active language; short greetings such as `Hi` / `Hello` or مرحبا can switch language.
Human handover	Patient can request reception; system stores context and may notify reception when configured. Safety wording can also escalate.
Appointment reminders	System schedules WhatsApp reminders (e.g. 24h and 2h before) and can process due reminders via CLI.
Admin dashboard	Clinic staff manage clinic data, doctors, hours, appointments, conversations, reminders, KPIs, and usage against production data.

Supporting operational requirements (from current design):

- Interactive WhatsApp UI (buttons, lists, cards; optional Flow) with free-text fallback for navigation and some matching.
- Protection against double booking (slot hold + row lock + unique active appointment per slot).
- Global navigation: `MENU` / قائمة and Back / عودة / رجوع / `Return`.
- Clinic holidays and doctor leave exclude non-bookable slots.

3. User Roles

Patient

Uses WhatsApp to talk to the clinic number. Can book, reschedule, and cancel appointments; search by specialty or doctor name; inquire about doctors/specialties; send voice notes; and request transfer to

reception. Interaction is guided by menus, lists, buttons, doctor cards, and optional WhatsApp Flow.

Receptionist

Receives cases that need human help. The bot creates a `HandoverCase` with conversation context. When `RECEPTIONIST_WHATSAPP` is configured, reception can be notified on WhatsApp. Handovers also appear in the admin dashboard conversations/handover views. Cutoffs that are too late for cancel/reschedule may also escalate to handover.

Admin / Clinic Staff

Uses the Streamlit Admin Dashboard (`dashboard/streamlit_app.py`) against the same PostgreSQL database to manage clinic profile and holidays, departments and doctors, working hours and leave, slot visibility, appointments and history, conversation transcripts and handovers, reminders, settings, KPIs, and usage/cost logs. Access is gated by dashboard login (`ADMIN_DASHBOARD_PASSWORD` / admin authentication in the dashboard service).

---

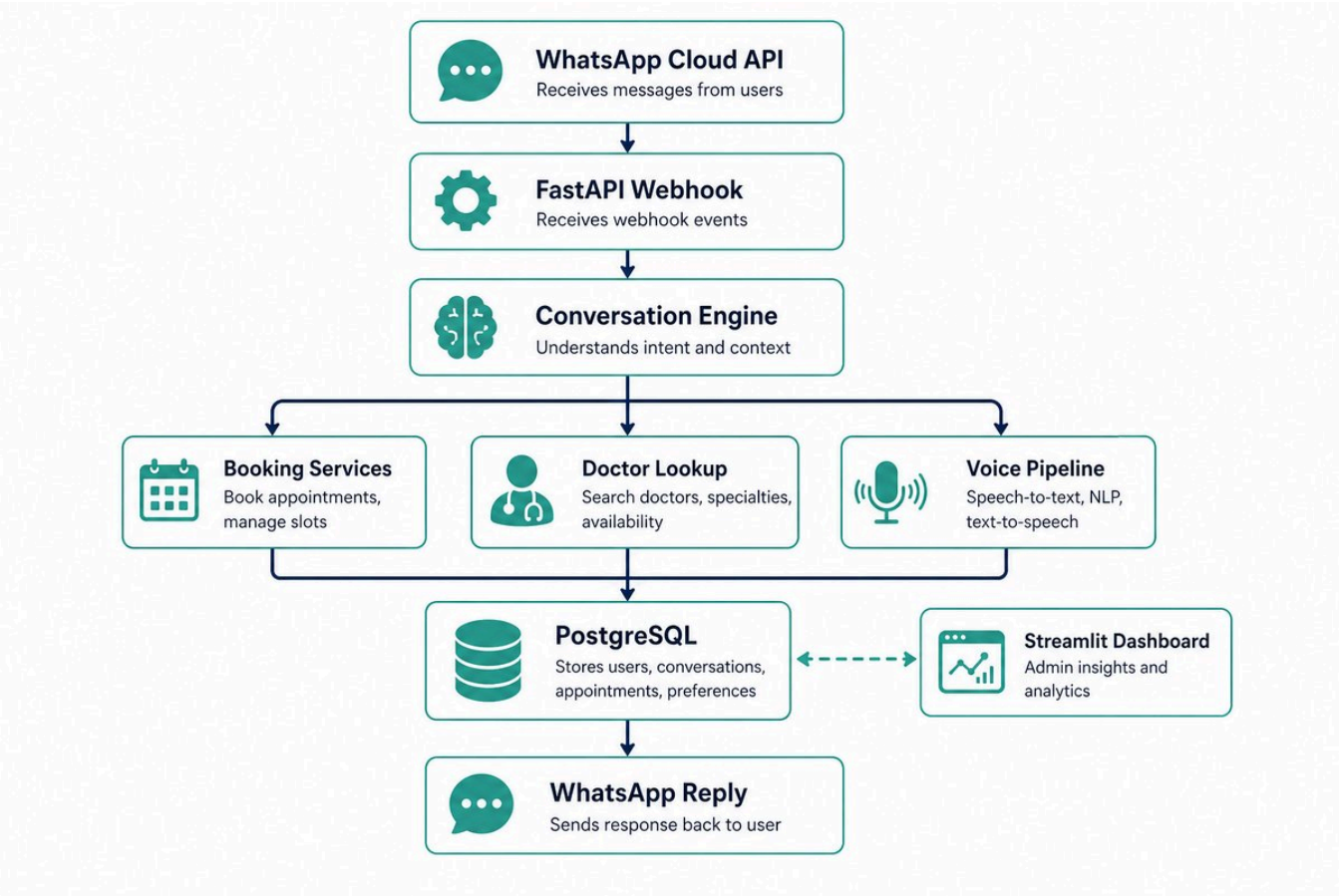
4. Technology Stack

Layer	Technology	Role
API	FastAPI + Uvicorn	Webhook + booking REST helpers
Database	PostgreSQL + SQLAlchemy 2 + `psycopg`	Patients, doctors, slots, appointments, conversations
Migrations	Alembic	Schema versioning (`alembic upgrade head`)
Conversation	LangGraph-style engine (+ `langchain-core` / `langchain-openai`)	Persist inbound → handle state
WhatsApp	Meta Cloud API	Inbound/outbound messaging
HTTP client	`requests`	Meta API calls and media download
Config	`python-dotenv`	Load `.env` settings
AI / Voice	OpenAI (Whisper STT + LLM structuring)	Voice understanding and ambiguous choice help
Language detect	`lingua-language-detector`	Arabic / English detection support
Schemas	Pydantic (`app/schemas/booking.py`)	Booking API request/response models
Uploads	`python-multipart`	FastAPI multipart support
Images	Pillow	Doctor Flow / card image handling
Dashboard	Streamlit + pandas	Ops and admin UI
Local tunnel	cloudflared (optional)	Expose localhost for Meta webhook testing

> This table is the **runtime architecture stack**. See also Appendix A.

---

5. System Architecture



High-level request flow

1. WhatsApp delivers an inbound message (text, interactive reply, or voice).
2. FastAPI webhook verifies/normalizes the payload.
3. Conversation engine loads session/conversation state for the phone.
4. Engine routes by intent and state (booking, reschedule, cancel, inquiry, voice, handover, menu).
5. Domain services read/write PostgreSQL.
6. Bot replies through WhatsApp.

Stage	Component
1	WhatsApp Cloud API
2	FastAPI webhook / API routes
3	Conversation engine
4a–4d	Booking / doctors / voice / handover services
5	PostgreSQL
6	WhatsApp outbound reply

Separation of responsibilities

Path	Responsibility
`app/api/routes`	HTTP + Meta webhook extraction
`app/graph`	Conversation state and interactive steps
`app/services`	Business logic
`app/models`	SQLAlchemy models
`app/schemas`	Pydantic API schemas
`app/core`	Config, i18n, safety, time helpers
`dashboard`	Admin UI
`alembic`	Migrations

Design notes

- Business rules live in **services**, not in the WhatsApp client.
- Interactive WhatsApp UI guides choices; free text still works for menu/navigation and some matching.
- Alembic owns schema changes on deploy.
- Local and server may use different DBs — treat the server dashboard as production truth.

Key files

File	Role
`app/main.py`	FastAPI app
`app/api/routes/webhook.py`	Meta webhook
`app/api/routes/booking.py`	Booking REST + Pydantic response models
`app/schemas/booking.py`	Booking API schemas
`app/graph/conversation_engine.py`	Conversation core (MENU / back, list pagination)
`app/services/patient_service.py`	Patient lookup + name validation
`app/services/choice_resolver.py`	Free-text → list option mapping (rules + LLM)
`app/language/language_detector.py`	AR/EN detection and switch gates
`app/services/booking_service.py`	Hold/confirm/reschedule/cancel
`app/services/slot_service.py`	Slot generation and locking
`app/services/doctor_service.py`	Departments/doctors/cards
`app/services/whatsapp_client.py`	Meta send helpers
`app/services/voice_*.py`	Voice pipeline
`app/services/flow_service.py`	Optional doctor Flow
`app/services/admin_dashboard_service.py`	Dashboard backend
`alembic/`	Migrations
`dashboard/streamlit_app.py`	Admin UI

6. System Workflow

End-to-end path of a typical patient request:

```
Patient
  → WhatsApp
  → WhatsApp Cloud API (Meta)
  → FastAPI webhook (GET verify / POST inbound)
  → Conversation engine (persist inbound → handle state)
  → Business services (booking, doctors, voice, handover, reminders, …)
  → PostgreSQL
  → WhatsApp outbound reply
```

Stage	Role
Patient	Sends text, taps a button/list row, opens Flow, or sends a voice note.
WhatsApp / Meta Cloud API	Delivers the event to the configured webhook URL and accepts outbound sends.
FastAPI webhook	Verifies the handshake (`GET /webhook`), accepts `POST /webhook` , classifies payload type, and schedules background handling with a DB session.
Conversation engine	Loads conversation/session for the phone; routes by state and intent; drives menus and steps.
Business services	Enforce booking rules, hold/confirm slots, manage patients/doctors, voice understanding, handover, reminders.
PostgreSQL	Persists org data, slots, appointments, conversations, messages, reminders, handovers, usage.
WhatsApp response	Bot sends text, buttons, lists, cards, or Flow CTAs back to the patient.

Admin operations follow a parallel path: **Admin** → **Streamlit dashboard** → **admin dashboard services** → **PostgreSQL** (same database as the bot).

7. Directory Structure

```
hospital_chatbot/
├── app/
│   ├── main.py                 FastAPI entry
│   ├── api/routes/             Webhook + booking REST
│   ├── graph/                  Conversation engine
│   ├── core/                   Config, i18n, safety, time
│   ├── language/               Language detection / prompts
│   ├── services/               Domain services
│   ├── models/                 SQLAlchemy tables
│   ├── schemas/                Pydantic models (e.g. booking)
│   ├── db/                     Engine, enums
│   ├── scripts/                Seed + reminders CLI
│   └── flows/                  WhatsApp Flow JSON
├── alembic/                    Migrations
├── dashboard/                  Streamlit admin UI
├── docs/                       This documentation + diagrams/PDFs + history
├── tests/                      Automated unit/integration-style tests
├── requirements.txt
├── .env.example
└── README.md                   Quick start only
```

8. Startup and API Entry Points

`app/main.py` creates the FastAPI app and mounts routers.

Endpoint	Purpose
`GET /`	Health/hello
`GET /webhook`	Meta verify handshake
`POST /webhook`	Inbound messages and statuses
`GET /booking/departments`	Specialty list
`GET /booking/doctors`	Doctor list
`GET /booking/doctors/{id}/slots`	Available slots
`POST /booking/confirm`	Confirm booking (API/testing)
`GET /docs`	Swagger UI

Booking REST responses use Pydantic models in `app/schemas/booking.py`.

On startup, `create_tables()` remains as a safe idempotent fallback (creates missing tables only). Prefer `alembic upgrade head` on deploy.

Full endpoint table: Appendix B.

---

9. WhatsApp Message Journey

1. Patient messages the business number.
2. Meta posts JSON to `POST /webhook` (`app/api/routes/webhook.py`).
3. Handler classifies text / button / list / Flow / audio / status.
4. Background task opens a DB session and calls `process_incoming_message`.
5. Graph runs: `node_persist_inbound` → `node_handle`.
6. Services update PostgreSQL; `whatsapp_client` sends the reply.

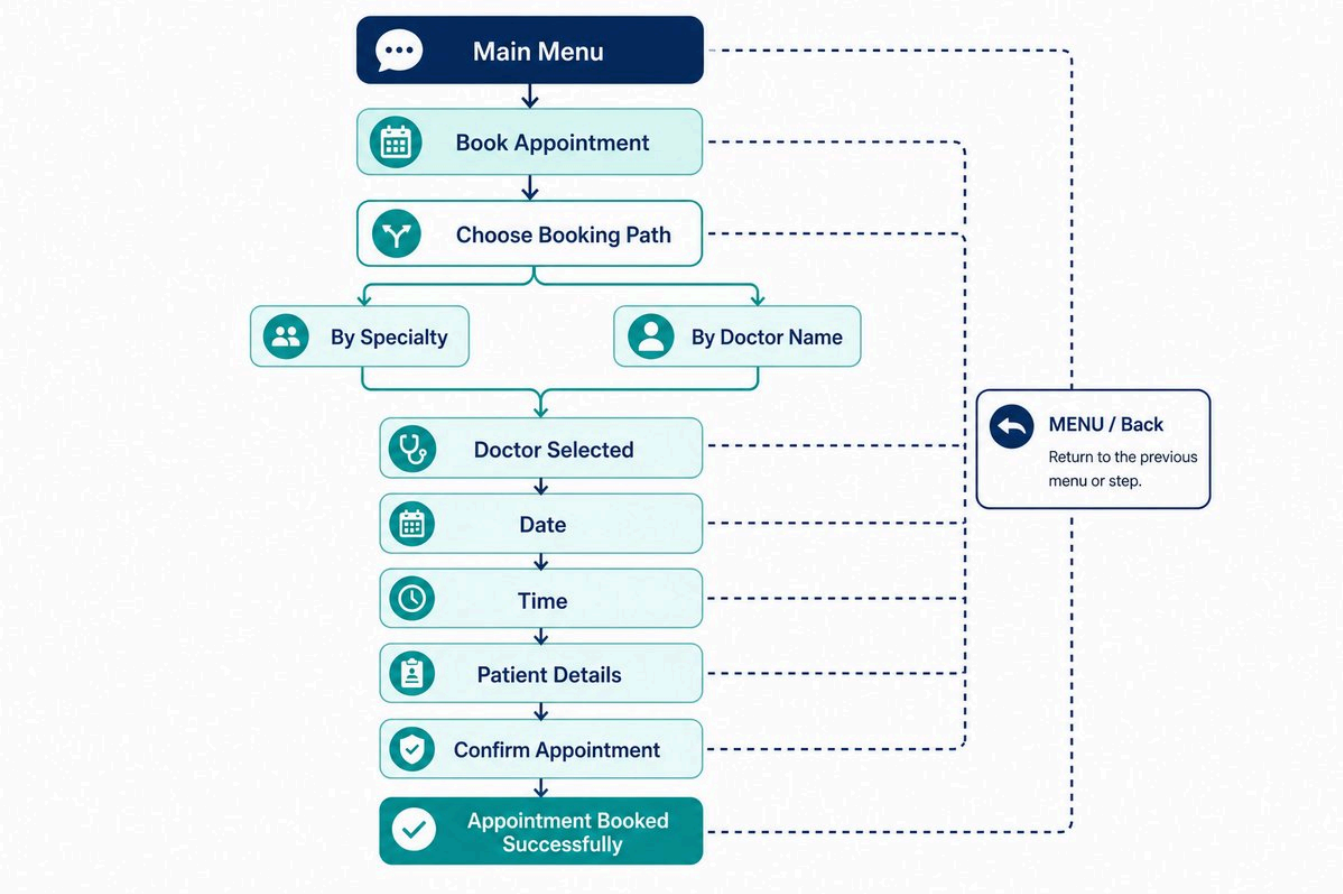
Conversation states (examples)

`new` · `language_select` · `main_menu` · `booking_doctor` · `booking_date` · `booking_slot` · `patient_details` · `confirming` · `reschedule` · `cancel` · `handover` · `closed`

Primary state is stored on `Conversation`. In-progress draft details also use an in-memory session map in the engine (with recovery/restart behavior after process reload).

---

10. Conversation Flows



Languages: Arabic and English — replies follow the patient's active language. Short greetings such as `Hi` / `Hello` or مرحبا switch language when the patient clearly opens in that language.

Main menu

#	Option
1	Book appointment
2	Reschedule appointment
3	Cancel appointment
4	Ask about doctors/specialties
5	Talk to reception

The main menu text also tells patients how to navigate (see below).

Global navigation

Action	How	Behavior
Main menu	Type `MENU`, `Menu`, قائمة, or منيو	Clears draft → main menu
Step back	Type عودة ,رجوع, `Back`, or `Return`	One sensible step backward

Notes:

- These commands work during active flows.
- Back is state-aware (time → date → doctor → path choice, etc.).
- Confirming a **cancellation** is separate from global back.

Book appointment

Path choice: By specialty **or** By doctor name.

By specialty: specialty list → doctor cards/options → shared steps.
By doctor name: paginated doctor name list → select doctor → shared steps.

WhatsApp list messages allow at most **10 rows**. Long lists (specialties, doctors-by-name, dates, times, upcoming appointments) use a trailing **More** / المزيد row when needed. Choosing More sends the **next page as a new list message** (WhatsApp cannot expand the same bubble in place).

Shared steps after doctor selected

Step	What happens
Date	Available dates (paginated if needed). Invalid typed day → ask to pick from the list and re-send dates
Time	Available slots + **Change day**; invalid typed time → ask to pick from the list and re-send times
Patient details	Reuse known name if available, or ask for full name; DOB only if `REQUIRE_DOB`
Confirm	Summary → confirm
Success	Appointment created + reference

Patient name

- First prompt asks for the patient's triple name with a short example.
- Validation requires at least three name parts, letters only (Arabic/English), optional internal hyphen (e.g. `El-Din`), each part at least two letters — no emoji, digits, or punctuation.
- On failure, a clearer validation message is sent and the patient can retry.

Free-text matching on lists

Typed replies can map to list options (rules, then LLM). For **date/time** lists the mapper is strict: questions like "is 7 available?" must not silently pick another slot; if the spoken clock hour is not in the chosen option, the bot rejects and re-offers the list.

Typical back (booking): doctor (specialty) → specialties; specialties/name list → path choice; date → doctor; time → date; details/confirm → time.

Reschedule

1. List upcoming confirmed appointments (or ask for ref).
2. Pick appointment → change type → new date/slot → confirm.
3. Old slot released; new slot booked; reminders refreshed.

Cancel

1. List upcoming appointments.
2. Confirm cancellation → slot released; reminders cancelled.

Inquiry / handover

- Inquiry: browse doctors/specialties; may continue into booking.
- Handover: acknowledge transfer; store phone/intent/context; notify reception when configured.

Voice (patient view)

Voice note → STT → intent/entities → validate availability → continue guided booking when possible.

---

11. Booking

Specialty / doctor selection

- Departments = specialties shown to patients (paginated when more than one WhatsApp page).
- **By doctor name** lists all active doctors in pages with **More** when needed (`list_active_doctors` + session offsets).
- Doctor cards via `doctor_card_payload` + `send_doctor_cards` (sequential cards, typically up to 5).
- Optional WhatsApp Flow carousel/radio via `flow_service` (fallback to cards on failure).
- Bulk demo data for local pagination tests: `python -m app.scripts.seed_bulk_test`.

Slots

- `get_available_dates_and_slots` validates doctor and releases expired holds.
- `ensure_slots_for_doctor` generates from weekly hours.
- Excludes past times, min lead time, leave, and clinic holidays.

Hold and confirm

1. `hold_slot` locks the row (`SELECT FOR UPDATE`), sets `held` (default ~3 minutes).
2. Patient details + summary confirm.
3. `confirm_booking` re-checks ownership/availability, creates/updates Patient + Appointment (`confirmed`), sets slot `booked`, writes history + reminders.

Double-booking protection

Layered: row lock + exclude booked slots + partial unique index on active appointments for a slot.

---

12. Reschedule and Cancel

- Ownership is tied to the WhatsApp phone.
- Cutoffs: `CANCEL_CUTOFF_HOURS` / `RESCHEDULE_CUTOFF_HOURS` (too late → handover).
- History rows and reminder cancellation/recreation are part of the transactional updates.

---

13. Working Hours, Slots, and Holidays

Concept	Model / behavior
Weekly hours	`DoctorWorkingHours` → generates slots
Doctor leave	`DoctorUnavailability` → overlapping slots `blocked`
Clinic holiday	`ClinicHoliday` → block slots across doctors

Slot status	Bookable?
`available`	Yes
`held`	No
`booked`	No (until cancel/reschedule)
`blocked`	No

14. Reminders

- Created at booking (default offsets e.g. 24h and 2h via `REMINDER_HOURS_BEFORE`).
- `process_due_reminders` sends due WhatsApp messages; dry-run supported.
- Cancel/reschedule clears pending reminders.
- Meta delivery receipts can update reminder status.
- CLI: `python -m app.scripts.run_reminders`

15. Voice

Current production path uses **OpenAI Whisper** (`OPENAI_STT_MODEL`, typically `whisper-1`) plus LLM structuring / Voice Router V2 when enabled.

1. Webhook receives `audio` + `media_id`.
2. Download from Meta → STT → optional hallucination scrub.
3. Understanding extracts intent/entities.
4. Executor validates availability and advances the guided flow.
5. Persist voice metadata / usage logs as configured.

> Voice is an alternate input into the **same** booking conversation system.

16. Safety and Handover

- `app/core/safety.py` detects high-risk medical wording; no diagnosis.
- Creates `HandoverCase` with context; may notify `RECEPTIONIST_WHATSAPP`.
- Unexpected webhook failures can escalate as high-priority handovers.

17. Data Structure & Database Design

High-level chains:

- Tenant → Clinic → Department → Doctor → WorkingHours / Unavailability / Locations / Slots
- Patient → Appointment → Slot → History + Reminders
- Conversation → Messages → VoiceMessage / HandoverCase
- Clinic → ConfigSetting / UsageLog / ClinicHoliday
- Tenant → AdminUser

Main entities and relationships

Entity	Table	Role / relationships (from models)
Tenant	`tenants`	Top-level org. Has clinics, doctors, admin users.
Clinic	`clinics`	Belongs to Tenant. Has departments, doctors, patients, conversations, appointments, slots, config settings, usage logs, holidays.
Department	`departments`	Belongs to Clinic. Has doctors (specialty).
ClinicHoliday	`clinic_holidays`	Belongs to Clinic. Blocks slots clinic-wide for a datetime range when active.
Doctor	`doctors`	Belongs to Tenant, Clinic, Department. Has locations, working hours, unavailabilities, slots, appointments.
DoctorLocation	`doctor_locations`	Belongs to Doctor. Practice location/room metadata.
DoctorWorkingHours	`doctor_working_hours`	Belongs to Doctor. Weekly schedule used to generate slots.
DoctorUnavailability	`doctor_unavailability`	Belongs to Doctor. Leave/blocked periods overlapping slots.
AppointmentSlot	`appointment_slots`	Belongs to Tenant, Doctor, Clinic. Unique on `(doctor_id, slot_start)`. Status: available / held / booked / blocked.
Patient	`patients`	Belongs to Tenant, Clinic. Unique on `(clinic_id, whatsapp_phone)`. Has appointments and conversations. Fields include name, optional DOB, gender, consent.
Appointment	`appointments`	Links Patient, Doctor, Clinic, Tenant, Slot. Has `appointment_ref`, status, reason, channel. Has history and reminders. Partial unique index prevents two active bookings on one slot.
AppointmentHistory	`appointment_histories`	Audit of appointment changes; can reference old/new slots.
Conversation	`conversations`	Per phone/clinic session state, language, intent. Has messages and handover cases; optional patient link.
Message	`messages`	Inbound/outbound WhatsApp messages for a conversation.
VoiceMessage	`voice_messages`	One-to-one with Message: media ref, transcript, STT provider, confidence, status.
Reminder	`reminders`	Belongs to Appointment. Scheduled send time, status, retries, WhatsApp message id.
HandoverCase	`handover_cases`	Belongs to Conversation. Reason, priority, assignee, status.
AdminUser	`admin_users`	Belongs to Tenant. Dashboard user email/role/active flag. Unique `(tenant_id, email)`.
ConfigSetting	`config_settings`	Belongs to Tenant + Clinic. Key/value clinic settings. Unique `(clinic_id, key)`.
UsageLog	`usage_logs`	Belongs to Tenant + Clinic. Provider/operation cost, tokens/minutes, latency, status.

Appointment statuses include: `draft`, `held`, `confirmed`, `rescheduled`, `cancelled`, `completed`, `no_show`, `pending_handover`.

> Full column-level field catalogs beyond the model modules are not duplicated here. Refer to `app/models/*.py` and Alembic migrations for authoritative schema.

18. Admin Dashboard

`dashboard/streamlit_app.py` (localized via `ui_i18n.py`).

Screen	Purpose
KPIs	Volume, cancel/reschedule, handovers, voice, cost
Clinic setup	Clinic profile + holidays
Departments / Doctors	Specialty and doctor CRUD
Working hours	Weekly hours, leave, slot generation
Doctor schedule	Free / booked / held / blocked
Appointments	List + status actions + history
Conversations	Transcripts + handovers
Reminders	Due send / dry-run / retry
Settings	Reminder/escalation/consent knobs
Usage / costs	Provider/operation spend

Official production URL: `http://196.219.86.38:8501/`

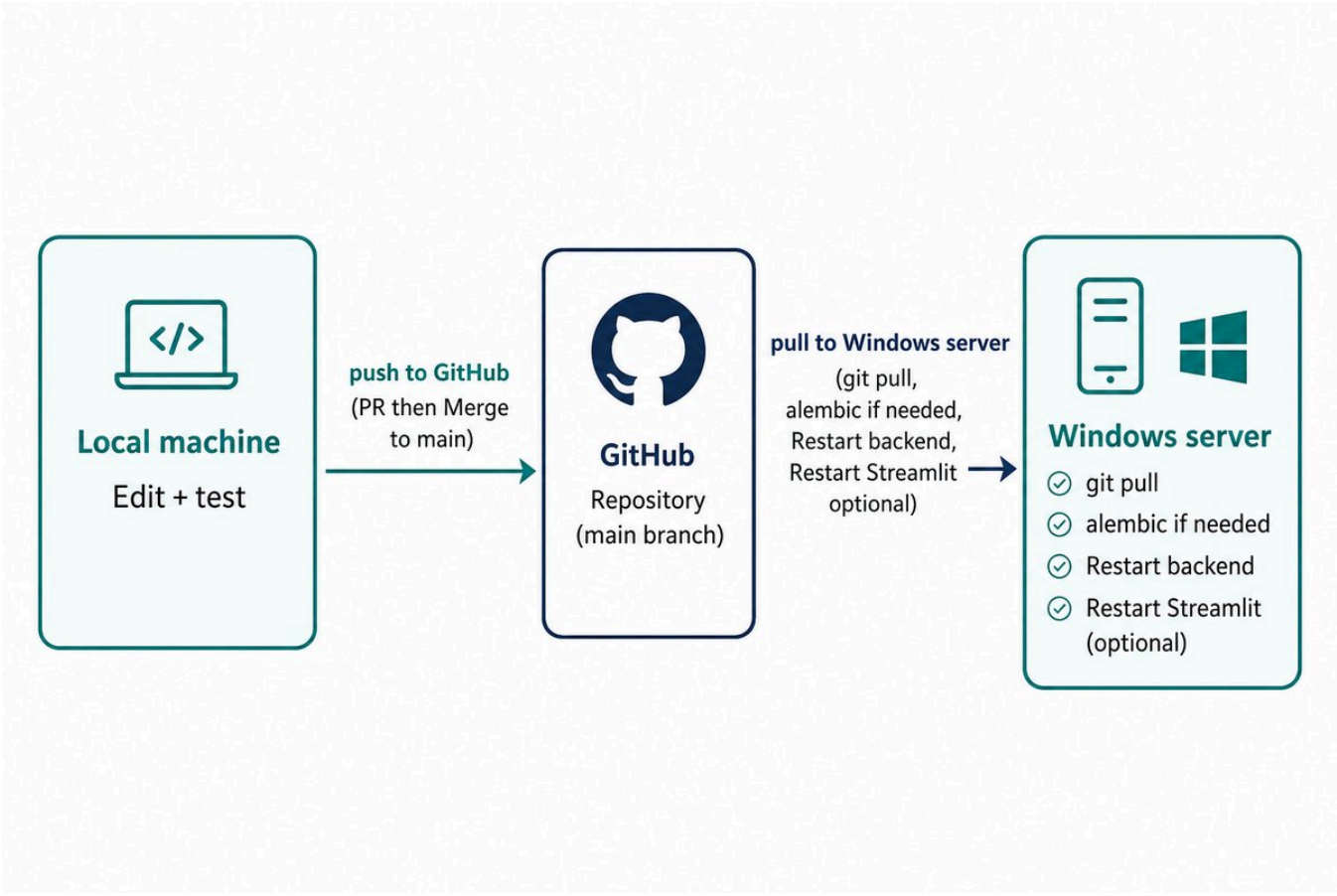
19. API Documentation

The HTTP surface is intentionally small: WhatsApp webhook plus booking REST helpers for testing without WhatsApp/OpenAI.

- Interactive OpenAPI/Swagger: `GET /docs` when the API is running.
- Pydantic request/response models: `app/schemas/booking.py`.
- Route implementations: `app/api/routes/webhook.py`, `app/api/routes/booking.py`.

See Appendix B for the endpoint list.

20. Deployment



Reliable path: merge to `main`, then update the Windows server **manually**.
GitHub Actions auto-deploy may fail (SSH auth).

Local operating commands

```
cd E:\4S\hospital_chatbot
pip install -r requirements.txt
.\venv\Scripts\python.exe -m alembic upgrade head
.\venv\Scripts\uvicorn.exe app.main:app --reload --host 0.0.0.0 --port 8000
.\venv\Scripts\streamlit.exe run dashboard/streamlit_app.py --server.port 8501
.\cloudflared tunnel --url http://127.0.0.1:8000
python -m app.scripts.run_reminders --dry-run --force
```

Local → GitHub

```
cd E:\4S\hospital_chatbot
git checkout -b <branch-name>
git add <files>
git commit -m "short clear message"
git push -u origin <branch-name>
```

Then PR → `main` → Merge.

Server — backend

Path: `C:\apps\hospital-chatbot`

```
cd C:\apps\hospital-chatbot
git checkout main
git pull
.\venv\Scripts\pip.exe install -r requirements.txt   # if requirements changed
.\venv\Scripts\python.exe -m alembic upgrade head
Restart-Service HospitalChatbotBackend
Get-Service HospitalChatbotBackend
Get-Content "C:\apps\hospital-chatbot\logs\stdout.log" -Tail 20 -Encoding UTF8
```

- First-time only for an already-complete DB: `alembic stamp head`, then always `upgrade head`.
- Services: `HospitalChatbotBackend`, `NgrokTunnel`.
  There is **no** `HospitalChatbotDashboard` Windows service.

Server — Streamlit

```
.\venv\Scripts\streamlit.exe run dashboard/streamlit_app.py --server.address 0.0.0.0 --server.port 8501
```

Keep the window open. Official URL: `http://196.219.86.38:8501/`

Environment	URL
Local dashboard	`http://localhost:8501`
Server dashboard	`http://196.219.86.38:8501/`
Local API	`http://127.0.0.1:8000`

WhatsApp Flow (doctor picker)

Optional ImageCarousel + radio list (not a marketing carousel template).

1. Install Pillow if needed: `pip install Pillow`
2. In Meta WhatsApp Manager → Flows → create custom Flow
3. Paste `app/flows/doctor_picker.json` → publish → copy Flow ID
4. `.env`:

```
USE_DOCTOR_FLOW=true
WHATSAPP_FLOW_ID=your_flow_id_here
WHATSAPP_FLOW_CTA=عرض الأطباء
```

5. Restart API (+ local tunnel if testing)
6. Book → specialty/doctors path → open Flow → swipe/select → booking continues

Notes:

- ImageCarousel max ~3 images (Meta); radio list can list more doctors.
- On Flow failure, bot falls back to sequential cards.
- Disable with `USE_DOCTOR_FLOW=false`.

Environment reference: Appendix C.

21. User Manual

This section is for **end users** (patients and reception/admin operators). It avoids backend implementation detail.

How to Book an Appointment

1. Start a WhatsApp conversation with the clinic number.
2. Choose **Book appointment** from the main menu (or type the equivalent).
3. Choose **by specialty** or **by doctor name**.
4. Select a specialty (if applicable), then select a doctor.
5. Select a date from the list (use **More** / المزيد if the list continues).
6. Select a time (or **Change day** to go back to dates).
7. Enter patient details when asked (full triple name; date of birth only if the clinic requires it).
8. Review the summary and confirm.
9. Receive the booking confirmation and reference.

Tips: Type `MENU` / قائمة to return to the main menu. Type رجوع / `Back` / `Return` to go one step back. Prefer picking from the list for dates and times.

How to Reschedule

1. Open WhatsApp and choose **Reschedule appointment**.
2. Select your upcoming appointment from the list (or provide the reference if asked).
3. Choose what you want to change (for example date/time/doctor, depending on options shown).
4. Pick the new date and time from the lists.
5. Confirm the change.
6. You should receive confirmation; previous reminders are replaced with new ones for the updated appointment.

If it is too close to the appointment time (clinic cutoff), the bot may transfer you to reception instead.

How to Cancel

1. Choose **Cancel appointment** from the main menu.
2. Select the upcoming appointment to cancel.
3. Confirm the cancellation when asked.
4. The appointment is cancelled and related reminders stop.

If cancellation is too late for the clinic policy, the bot may hand you over to reception.

How to Use Voice Messages

1. Record and send a WhatsApp voice note to the clinic number.
2. The bot converts speech to text and tries to understand the request (book, change day, specialty, and so on).
3. When possible, it continues the same guided booking/reschedule flow (lists and confirmations).
4. If understanding is unclear, follow the on-screen buttons or typed prompts the bot sends next.

How to Contact Reception

1. Choose **Talk to reception** from the main menu, or follow a handover prompt if the bot cannot complete a sensitive or late change.
2. The bot acknowledges the transfer and stores your conversation context for staff.
3. A receptionist may contact you (when reception WhatsApp notification is configured). Staff can also review handovers in the admin dashboard.

Admin Dashboard (staff)

1. Open the dashboard URL (local or production server).
2. Sign in with the configured admin credentials.
3. Use the screens for clinic setup, doctors/hours, appointments, conversations/handovers, reminders, KPIs, and usage as needed.

---

22. Testing

Automated tests in the repository

The `tests/` package currently focuses mainly on **voice routing helpers**, specialty word matching, and related cost/language utilities. Examples of covered areas:

Area	Evidence in repo
Specialty word / alias matching	`tests/test_specialty_word_match.py`
Voice router V2 / state-aware routing	`tests/test_voice_router_v2.py`, `tests/test_voice_state_aware.py`
Voice change-target / change-day behavior	`tests/test_voice_change_target.py`
Voice date/time hints	`tests/test_voice_date_time_hints.py`
Voice reply language	`tests/test_voice_reply_language.py`
Voice cost / usage estimation	`tests/test_voice_cost.py`
Local pagination demo data	`python -m app.scripts.seed_bulk_test` (seed script, not a pytest suite)

Manual / product scenarios

Booking, reschedule, cancel, Arabic/English conversation, invalid list input, slot conflicts, double-booking protection, handover, and reminders are exercised in the live WhatsApp/API flows described in this document. A formal, versioned QA results matrix for those scenarios is **not stored in the repository** — [Needs Verification] if stakeholders require a signed test report.

There is no claim in this document that every product scenario above has an automated pass/fail record in CI.

---

23. Conclusion

Hospital Chatbot provides a production-oriented WhatsApp channel for clinic appointment booking and management, with Arabic/English support, voice input, receptionist handover, reminders, and a Streamlit

operations dashboard on a shared PostgreSQL database.

Technical ownership is separated across FastAPI routes, a LangGraph-style conversation engine, domain services, and Alembic-managed schema.

---

24. Document History

This section records **changes to this documentation file only** (not the product/code timeline). Add a new row when the guide structure or major content is updated.

Date	Change
2026-07-30	README updated with features, quick start, and links to project docs.
2026-08-01	Unified project guide created as `DOCUMENTATION.md` with diagrams and PDF export. Earlier separate guides (`ARCHITECTURE`, `FLOWS`, `DEPLOY`, `KNOWN_ISSUES`, `WHATSAPP_FLOW_SETUP`) were folded into this single document.
2026-08-02	Documentation expanded for stakeholders: Project Requirements, User Roles, System Workflow, Data Structure & Database Design, User Manual, Testing, Conclusion, and Appendix (tools, API endpoints, environment variables). Technology stack table clarified for runtime components. Intro metadata box, test checklist, and Known Issues section removed from earlier drafts of this guide.
2026-08-02	Document History restored as section 24 (documentation changelog only). Cursor Pro / Chatbot described as development and research assistants (not runtime). Legacy prototype and unused-package notes removed from the guide text.

When you update this guide later, append one line above with the date and a short description of what changed in the **document**.

---

25. Appendix

Appendix A — Tools & Technologies

Cursor Pro and **Chatbot** are development/research assistants only. They are **not** part of the runtime architecture or production stack.

Production Technologies

Tool / Technology	Category	Purpose	Where It Is Used
FastAPI	API framework	HTTP API and webhook hosting	`app/main.py`, `app/api/routes`
Uvicorn	ASGI server	Runs the FastAPI app	Local and Windows service
PostgreSQL	Database	Persistent clinic and chat data	Via `DATABASE_URL`
SQLAlchemy 2	ORM	Models and sessions	`app/models`, `app/db`
psycopg	DB driver	PostgreSQL connectivity	SQLAlchemy engine
Alembic	Migrations	Schema versioning	`alembic/`
LangGraph	Conversation orchestration	Persist inbound → handle	`app/graph/conversation_engine.py`
langchain-core / langchain-openai	LLM tooling	Supporting LangGraph / OpenAI integration	Conversation / AI paths
Meta WhatsApp Cloud API	Messaging platform	Inbound/outbound WhatsApp	`whatsapp_client`, webhook
requests	HTTP client	Meta API and media download	WhatsApp, voice, flow services
OpenAI (GPT + Whisper)	AI / STT	Structuring, choice help, transcription	Voice + choice resolver
Pydantic	Schemas	Request/response validation	`app/schemas/booking.py`
python-dotenv	Config	Load environment variables	`app/core/config.py`
python-multipart	Uploads	Multipart form support	FastAPI stack
Pillow	Imaging	Doctor Flow / image handling	`flow_service`
lingua-language-detector	NLP	Language detection support	`language_detector`
Streamlit	Admin UI	Operations dashboard	`dashboard/streamlit_app.py`
pandas	Dataframes	Dashboard tables/data	Dashboard
cloudflared	Tunnel (optional)	Local webhook exposure	Local ops

Development Tools

Tool / Technology	Category	Purpose	Where It Is Used
Git / GitHub	Source control	Versioning, PRs, Actions	Repository + CI workflow
GitHub Actions	CI/CD	Deploy workflow to Windows server	`.github/workflows` (SSH may fail)
pytest / `tests/`	Automated testing	Voice and specialty unit tests	`tests/`
Alembic CLI	Schema ops	`upgrade` / `stamp` / revisions	Deploy and local
Swagger UI (`/docs`)	API exploration	Interactive API docs	Running FastAPI app
PowerShell	Ops scripting	Local and server commands	Windows host
Cursor Pro	IDE AI assistant	Writing and improving code, fixing bugs, and developing the system	Developer workstation (not runtime)

Research & Productivity Tools

Tool / Technology	Category	Purpose	Where It Is Used
Chatbot	Research / problem-solving	Finding solutions and researching topics when needed	Offline research (not runtime)
Project documentation / PDF export	Docs	`DOCUMENTATION.md`, diagrams, `_export_pdf.py`	`docs/`

Appendix B — API Endpoints

Method	Endpoint	Purpose
`GET`	`/`	Health / hello payload listing main route groups
`GET`	`/webhook`	Meta WhatsApp verify handshake
`POST`	`/webhook`	Inbound WhatsApp messages and delivery statuses
`GET`	`/booking/departments`	List specialties/departments for a clinic
`GET`	`/booking/doctors`	List doctors (optionally filtered by department)
`GET`	`/booking/doctors/{doctor_id}/slots`	Available dates/slots for a doctor
`POST`	`/booking/confirm`	Confirm a booking (REST/testing path)
`GET`	`/docs`	OpenAPI Swagger UI
`GET`	`/openapi.json`	OpenAPI schema ([Needs Verification] if customized beyond FastAPI defaults)

Query/body parameters follow `app/api/routes/booking.py` and `app/schemas/booking.py` (for example `clinic_id`, `department_id`, confirm payload fields).

Appendix C — Environment Variables

See `.env.example` for the full template. **Never commit real `.env` secrets.**

Group	Examples	Purpose
Database	`DATABASE_URL`, `DEFAULT_TENANT_ID`, `DEFAULT_CLINIC_ID`	DB connection and MVP single-clinic defaults
WhatsApp	`VERIFY_TOKEN`, `WHATSAPP_TOKEN`, `PHONE_NUMBER_ID`, `WHATSAPP_API_VERSION`	Meta Cloud API auth and webhook verify
Flow (optional)	`USE_DOCTOR_FLOW`, `WHATSAPP_FLOW_ID`, `WHATSAPP_FLOW_CTA`, `WHATSAPP_FLOW_DRAFT`	Doctor picker Flow
AI / voice	`OPENAI_API_KEY`, `OPENAI_MODEL`, `OPENAI_STT_MODEL`, `OPENAI_VOICE_STRUCTURE_MODEL`, `VOICE_ROUTER_V2`, `VOICE_STT_PROMPT`, `VOICE_REQUIRE_CONFIRM`, `VOICE_SKIP_CONFIRM_THRESHOLD`, `VOICE_LANG_SESSION_MINUTES`	LLM + Whisper + voice policy
Optional AI key	`GROQ_API_KEY`	Present in config/ `.env.example`; active runtime use [Needs Verification]
Booking policy	`SLOT_HOLD_MINUTES`, `BOOKING_WINDOW_DAYS`, `MIN_LEAD_HOURS`, `CANCEL_CUTOFF_HOURS`, `RESCHEDULE_CUTOFF_HOURS`, `DUPLICATE_APPOINTMENT_DAYS`, `ALLOW_OVERBOOKING`	Holds, windows, cutoffs
Patient fields	`REQUIRE_DOB`	Ask for date of birth when true. `REQUIRE_MRN` appears in older doc notes but is **not** present in `.env.example` / `app/core/config.py` — [Needs Verification]
Reminders	`REMINDER_HOURS_BEFORE`	Comma-separated hours before appointment
Clinic copy	`CLINIC_DISPLAY_NAME`, `CLINIC_LOCATION`, `CLINIC_LOCATION_AR`, `CLINIC_MAPS_URL`	Patient-facing clinic text
Escalation	`RECEPTIONIST_WHATSAPP`	Reception notification number
Dashboard	`ADMIN_DASHBOARD_PASSWORD`	Streamlit dashboard gate (used in dashboard code; not listed in `.env.example`)
Logging	`LOG_LEVEL`	App log verbosity (`app/main.py`)